# AEGIS USER GUIDE

*A guide for the engineering team*

28 October 2026 · Companion to AEGIS_TECHNICAL_DOCUMENTATION

AEGIS is a software protection platform. We are building it to make software harder to understand, copy, modify, or reconstruct through reverse engineering, and to measure how much the protection helps.

Our intended scope includes desktop software, websites and web applications, mobile apps, games, server software, embedded systems, and authorized console builds. Different environments need different protection methods. The platform brings those methods into one workflow.

This guide explains the project, your assignment, and how to begin. The Technical Documentation expands the same design into interfaces, build instructions, experiments, and detailed implementation briefs. Read this guide first, then the sections linked from your assignment.

## What the developer should be able to do

A developer chooses the part of their software they want to protect and the costs they can accept. AEGIS applies a compatible method, checks that the software still works, and records the result. The planned Adversarial Lab then attempts to reconstruct the protected logic under controlled conditions.

For example, a game studio could protect selected gameplay routines, compare frame time before and after protection, and test whether an analysis agent can recover the same behavior. A web team would need a browser-compatible method and browser tests. The goal is the same; the build path is different.

## The parts we are building

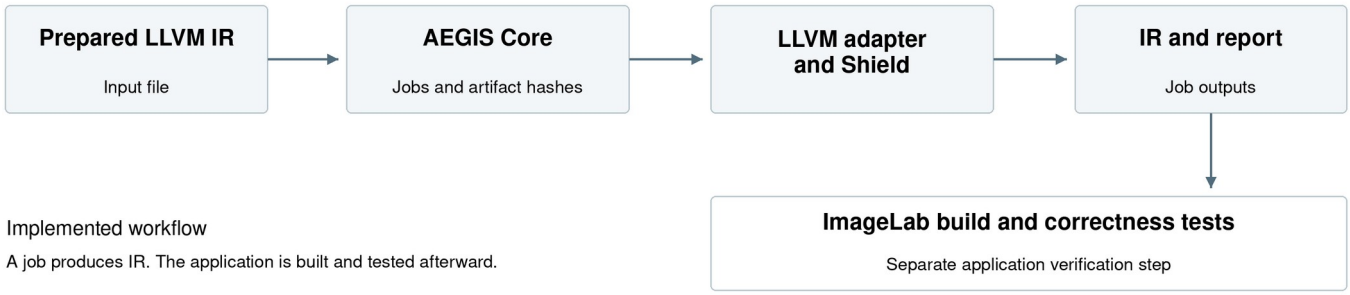
| Part | Its purpose |
|---|---|
| Shield | Transform selected program logic while preserving its intended behavior. |
| Mirage | Add harmless decoys and test whether they make analysis less effective. |
| Signature | Sign build records and investigate recoverable software watermarks. |
| Sentinel | Attach machine-readable rights and usage policies for cooperating tools. |
| Adversarial Lab | Run authorized reconstruction attempts and score the results. |
| Platform | Connect the tools, keep artifacts and reports, and later compare protection options. |

**The next proof we need:** a fair comparison showing whether a protected build is harder to reconstruct than its unprotected counterpart, with correctness and performance measured alongside the result.

Deeper reading: Technical Documentation 01 and 02.

# What works today

The working foundation is a local command-line tool, an HTTP API, a job runner, and an LLVM protection backend. LLVM is a compiler toolkit; its intermediate representation, or IR, is the form of code our first backend accepts.




The present adapter takes a textual LLVM IR file, applies Shield, and returns transformed IR plus a report. Building the final application and checking its behavior are separate steps today. The intended product will bring these steps together.

## The evidence we have

| Recorded result | What it tells us |
|---|---|
| 199 Python tests passed | The recorded combined run covered Core, API, CLI, and the real LLVM adapter. |
| 43 arithmetic sites across 3 ImageLab functions | The recorded application job applied the current Shield transformation. |
| 11 of 11 ImageLab tests passed with external IR | The protected output could be rebuilt and passed that configuration's correctness checks. |

These are earlier recorded results, not new tests run for this document. The code snapshot is **c995857**. The current transformation has no established resistance to reverse engineering, and later compiler optimization may simplify it away.

Shield v2, Mirage, Signature, Sentinel, the real Adversarial Lab, the optimizer, and the dashboard remain work to build. Core already has authorization and result contracts for Lab jobs; that does not mean a real attack backend exists.

## Why we use more than one test application

**ImageLab** is our small C++ test application. It makes defects easy to isolate and has independent checks for expected output. Keep it as a regression tool.

**PhotoCraft, reVC, and Mattermost** are the requested external targets: a Rust desktop editor, a native game, and a Go server with a web client. They test different integration problems. None is currently demonstrated as protected by AEGIS.

The engine must work from supported code structures and formats, not application names or memorized answers. Reserve additional, unseen programs for final evaluation. Supporting three examples will not establish support for all software.

Deeper reading: Technical Documentation 05 through 08 and the evidence register in 06.

# Your assignment

There are six workstreams. Youssef leads integration; the remaining roles need named teammates. The numbers below match GitHub issues 1 through 6 and Technical Documentation 10. Proposed new folders must be confirmed in the repository ownership rules before implementation.

## 1 Integration and release

Make the existing system easy to build, test, and combine. Start with clean-clone CI for Python, the LLVM plugin and adapter, and ImageLab. Update the stale ownership and architecture files, then connect each team's deliverable through the existing contracts.

**First deliverable:** a pull request with working CI, current contributor instructions, and a repeatable end-to-end runbook. The adaptive optimizer comes after trustworthy measurements exist.

**Done when:** another teammate can reproduce the build and application test from a clean checkout, required CI jobs actually execute, and failures appear accurately in the report.

Own shared Core/API/CLI integration, repository governance, and ImageLab integration. Coordinate contract changes with affected owners. Issue 1 · Technical Documentation 10.1.

## 2 Shield v2

Improve the protection of native code beyond the current integer-addition rewrite. First measure what remains after normal optimization. Then implement one additional transformation with a clear explanation of why it preserves program behavior.

**First deliverable:** a tested transformation, final binary inspection, and runtime and size comparisons against both the matching IR baseline and an ordinary release build.

**Done when:** selected and unselected code behave correctly, edge cases and rejection paths are tested, and the report distinguishes applied IR changes from effects retained in the final program.

Own `protectors/llvm/`, including the adapter and its local schemas. Agree any shared protocol change with integration. Issue 2 · Technical Documentation 10.2.

## 3 Adversarial Lab

Build the experiment that tests our central claim. Give an analysis tool and a reconstruction model equivalent access to baseline and protected artifacts. Compile their candidate implementations in isolation and score them with tests the model cannot see.

**First deliverable:** one reproducible ImageLab comparison using Ghidra and a Nebius-hosted model, with fixed budgets, saved evidence, and independent scoring.

**Done when:** both authorization checks remain enforced, each attempt is tied to an artifact hash, and failures, errors, and inconclusive trials are reported honestly.

Proposed ownership: `attack_lab/`. Coordinate schemas with integration and target builds with Shield. Issue 3 · Technical Documentation 10.3.

# Your assignment continued

## 4 Mirage

Investigate whether harmless decoy code changes an analyst's choices. Start with one deterministic decoy technique and a build with the technique disabled. Check that the decoy survives compilation and that both builds still produce the same results.

**First deliverable:** a decoy-enabled build, a matching control, retention measurements, and runtime and size costs. Use the Lab to measure whether the decoy affects reconstruction or consumes more of the analysis budget.

**Done when:** there are no unintended behaviors or external actions, functional tests pass, and any effectiveness claim comes from a comparison. A decoy that has no measured benefit is still a valid research result.

Proposed ownership: `protectors/mirage/`. Shared LLVM edits belong with the Shield owner. Issue 4 · Technical Documentation 10.4.

## 5 Signature and Sentinel

Make it possible to verify who signed a build record and whether the recorded artifact has changed. Start with an offline signed manifest. Add a machine-readable rights policy, then investigate a small watermark as a separate experiment.

**First deliverable:** manifest creation and verification, public-key verification instructions, tamper tests, and a versioned policy format. Keep private keys outside the repository.

**Done when:** modified artifacts and manifests fail verification, supported policy files validate, and the report explains the difference between a signature, a watermark, and a policy notice. A policy notice alone cannot force another tool to comply.

Proposed ownership: `provenance/` and `sentinel/`. Coordinate artifact linkage with integration. Issue 5 · Technical Documentation 10.5.

## 6 Dashboard and demonstration

Build the interface that lets someone submit a real job and understand its outcome. Use the existing API for job creation, status, and reports. Show missing tools and failed jobs as clearly as successful ones.

**First deliverable:** one complete UI flow backed by a real LLVM job, followed by a reproducible ImageLab walkthrough. Request API support for downloads or job listing where needed; those routes do not exist yet.

**Done when:** the displayed identifiers, hashes, results, and errors come from actual jobs, and a teammate can follow the demo without special local knowledge.

Proposed ownership: `platform/dashboard/` and presentation assets. Integration owns changes to ImageLab's code and build harness. Issue 6 · Technical Documentation 10.6.

## How the work meets

Shield and Mirage produce candidate builds. The Lab evaluates them. Signature links signed records to specific artifacts. The dashboard presents the evidence. Integration maintains the shared workflow and resolves changes that affect more than one module.

# Get started

## 1 Choose a bounded first task

Read your GitHub issue and Technical Documentation 10. Record your workstream, first deliverable, owned paths, and acceptance tests. Follow the current `AGENTS.md`, `ARCHITECTURE.md`, and module README. If their ownership descriptions conflict, ask the integration lead to reconcile them before editing shared files.

## 2 Create your working branch

For a new checkout, these commands work in Bash and Fish. Replace `your-area` with your workstream name. If you already have a checkout, inspect its status before switching branches.

```
git clone https://github.com/Youssef-Boussabah/AEGIS.git
cd AEGIS
git switch dev/integration
git switch -c feat/your-area
git rev-parse HEAD
```

The documented development baseline is Linux x86-64, Python 3.11 or newer, and matching LLVM/Clang 22.x tools. The recorded runs used Python 3.14.7 and LLVM 22.1.8. See Technical Documentation 09 for the complete setup and Shield/ImageLab commands.

```
uv venv .venv
uv pip install --python .venv/bin/python \
  -c core/requirements-dev.lock \
  -e './core[dev]' -e './platform/api[dev]' \
  -e './platform/cli[dev]'
.venv/bin/pytest
```

That last command runs the three Python package suites. The real LLVM adapter suite needs its own paths and tool arguments; it is not included in the default root test selection.

## 3 Reproduce before changing

Run the tests relevant to your module and save the command and result. Build one small working change, then test the intended behavior and its failure cases. If a tool is missing, say the test was not run. Keep `.aegis/`, generated builds, tokens, and private keys outside commits.

## 4 Hand over something reviewable

Your pull request should explain what changed, which files changed, how to run it, which tests passed or failed, and what remains unresolved. Target `dev/integration`. Follow the repository's rules on when to commit, push, and open a PR; using an agent does not grant it extra permissions.

**First-day outcome:** a working environment, reproduced baseline tests, a scoped implementation plan, and agreement on any dependency that blocks your first deliverable.

# Work with your coding agent

Use whichever coding agent you prefer. Give it both documents, the repository, and your issue. Fill in the assignment below. The full working brief in Technical Documentation 12 adds the detailed contracts and handoff format.

## Copy this brief

You are helping implement AEGIS, a software protection platform intended for desktop software, web applications, mobile apps, games, server software, embedded systems, and authorized console builds. We use different backends for different execution environments. Our goal is to preserve intended behavior while increasing the measured difficulty of reverse engineering and reconstruction.

My workstream is **[number and name]**. My issue is **[link]**. My allowed paths are **[paths]**. The first deliverable is **[specific behavior]**. Its acceptance tests are **[checks]**. Repository actions I authorize are **[edit and test only, or named additional actions]**.

Read AEGIS_USER_GUIDE, the relevant sections of AEGIS_TECHNICAL_DOCUMENTATION, AGENTS.md, ARCHITECTURE.md, and the module README. Inspect the current branch, commit, worktree, and code before relying on a document's status claim. Explain any mismatch.

The documented baseline contains Core, API, CLI, a textual LLVM 22 adapter, a prototype arithmetic transformation, and ImageLab. The adapter returns IR and a report; compiling and testing the application is a separate step. Shield v2, Mirage, Signature, Sentinel, the real Lab, optimizer, dashboard, and other backends are planned unless current code and tests establish otherwise.

Start by summarizing the relevant implementation, proposing a small change, identifying affected interfaces, and naming the tests you will run. Then implement within my assigned scope. Keep shared models, adapter protocols, and schemas compatible; route necessary changes through their owners.

Preserve program behavior, artifact hashing, path checks, subprocess limits, and both Lab authorization checks. Use independent expected results where possible. Inspect final compiled artifacts when evaluating transformations. Keep development tests separate from hidden evaluation data.

ImageLab, PhotoCraft, reVC, and Mattermost are test targets. Do not add application-specific protection rules or tune against hidden answers. Record tools, model identity, settings, budgets, hashes, and failed trials. Do not invent measurements or treat successful transformation as proof of security.

Finish with the files changed, exact test commands and outcomes, remaining limitations, contract questions, and the next concrete step. Write a short handoff note tied to the commit so another engineer or agent can continue.

## Keep the shared understanding current

After an accepted change, update the affected module documentation and record important decisions. Handoffs should live with the repository. A conversation is useful working context; it is not the only record of what the team has built.

Repository and source index: Technical Documentation 13. Both documents share the 28 October 2026 baseline at c995857.